hashlock.

Security Audit

Cartha (DeFi)

January 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

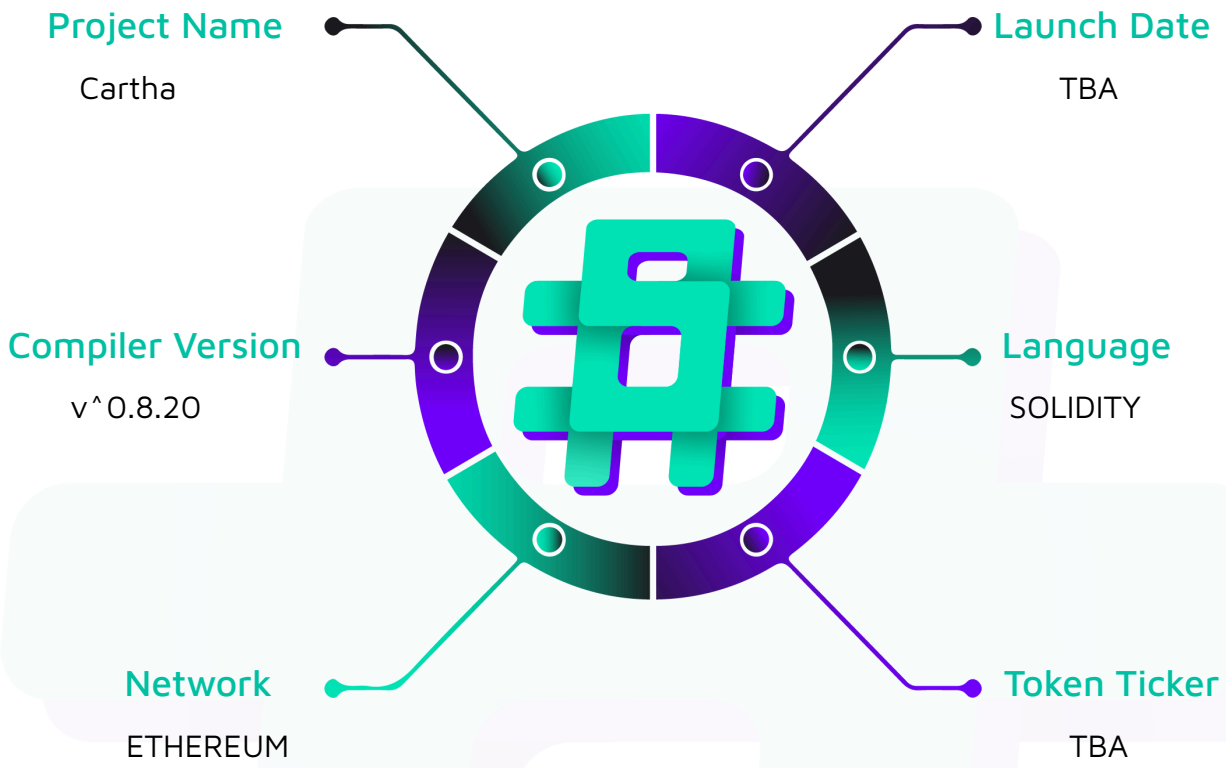
The Cartha team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

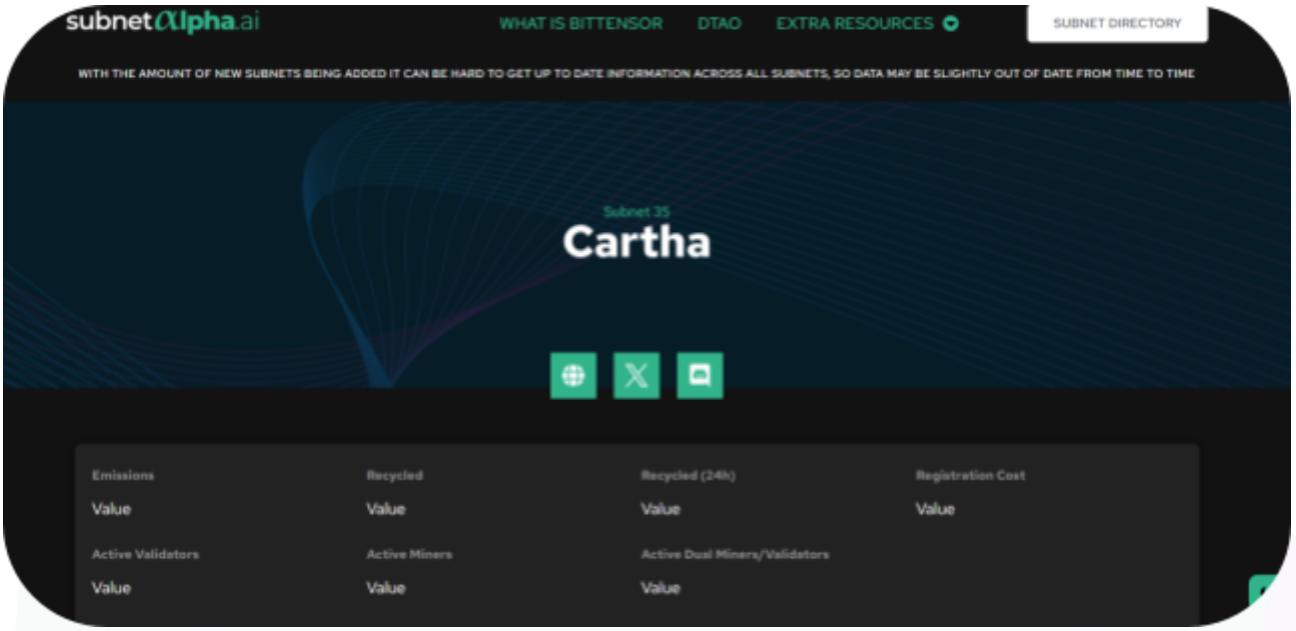
Cartha is a decentralized liquidity engine operating as Subnet 35 on the Bittensor network, designed to power the 0xMarkets decentralized exchange (DEX) by coordinating liquidity provision in a permissionless way. Liquidity providers ("miners" in Bittensor terminology) deposit USDC into on-chain vaults, and Cartha's protocol dynamically allocates that liquidity across trading pairs (forex, crypto, commodities) based on real-time demand. This enables deep, transparent liquidity and high-leverage perpetual futures trading on 0xMarkets without centralized intermediaries, while incentivizing capital commitment with trading fee shares and native token rewards.

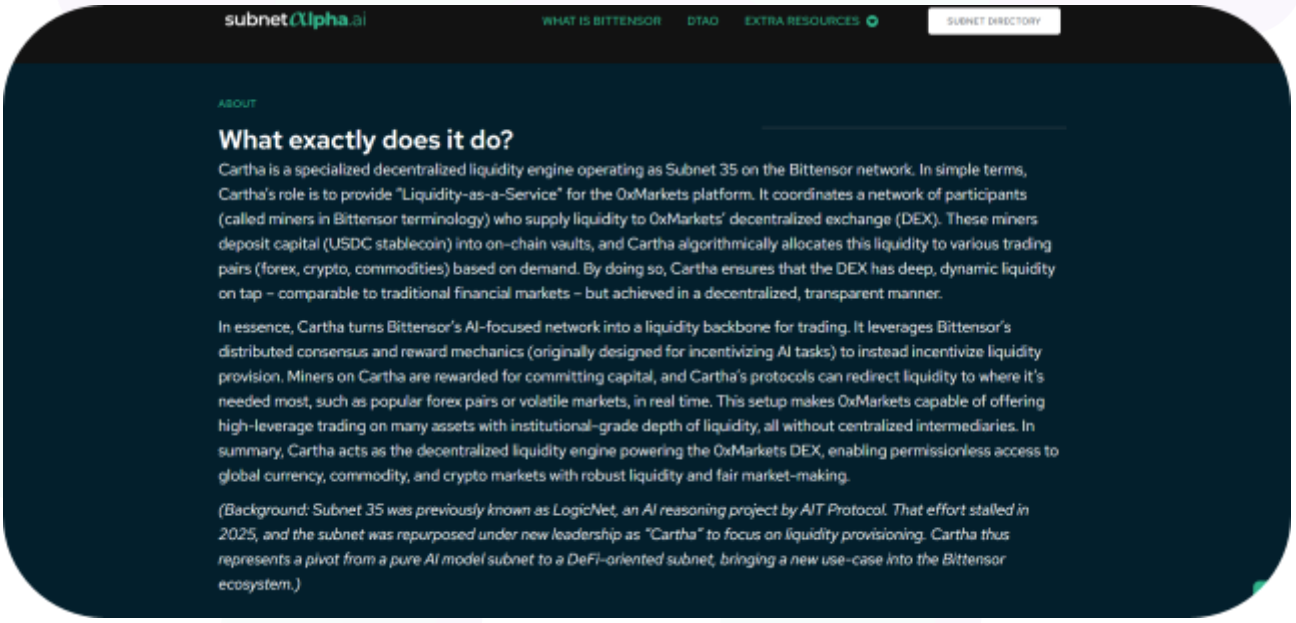
Project Name: Cartha
Project Type: Defi
Compiler Version: ^0.8.20
Website: https://subnetalpha.ai/subnet/cartha/
Logo:

Visualised Context:



Project Visuals:





Audit Scope

We at Hashlock audited the solidity code within the Cartha project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Cartha Smart Contracts
Platform	**Ethereum / Solidity**
Audit Date	**January, 2026**
Contract 1	CarthaParentVault.sol
Contract 2	CarthaVault.sol
Contract 3	CarthaVaultFactory.sol
Contract 4	LibDiamond.sol
Contract 5	Diamond.sol
Contract 6	DiamondInit.sol
Contract 7	Roles.sol
Contract 8	AccessControlFacet.sol
Contract 9	DiamondCutFacet.sol
Contract 10	DiamondLoupeFacet.sol
Contract 11	ICarthaParentVault.sol
Contract 12	ICarthaVault.sol
Contract 13	ICarthaVaultFactory.sol
Contract 14	ICarthaAccessControl.sol
Contract 15	IDiamondCut.sol
Contract 17	IDiamondLoupe.sol
Contract 18	ICarthaParentVaultErrors.sol

Contract 19	ICarthaVaultErrors.sol
Contract 20	ICarthaVaultEvents.sol
Audited GitHub Commit Hash	d9ac8f457fbb5d2aa68115d6d8656a9a2ab4fb23
Fix Review GitHub Commit Hash	5acf1558abd2cb6654a0b9bdfd1cdf4406e73977

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

6 Medium severity vulnerabilities

5 Low severity vulnerabilities

5 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
CarthaParentVault.sol - `CarthaParentVault` holds shared rules and admin controls for all vaults.	**Contract achieves this functionality.**
CarthaVault.sol - `CarthaVault` keeps user tokens safe and lets users deposit and withdraw.	**Contract achieves this functionality.**
CarthaVaultFactory.sol - `CarthaVaultFactory` makes new `CarthaVault` instances and links them to the parent.	**Contract achieves this functionality.**
Diamond.sol - `Diamond` is the main contract that routes calls and lets the system be upgraded.	**Contract achieves this functionality.**
Facets - `AccessControlFacet`, `DiamondCutFacet`, `DiamondLoupeFacet` manage permissions, upgrades, and give info about the diamond.	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the Cartha project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Cartha project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] CarthaParentVault#deposit, rebalance - Parent Vault Denial of Service on Child Pause

Description

`CarthaParentVault` suffers from a complete Denial of Service (DoS) on `deposit` and `rebalance` operations if a single underlying `CarthaVault` is paused. The parent contract iterates all child vaults atomically, meaning a revert in one child blocks the entire system.

Vulnerability Details

`CarthaParentVault` manages funds by calling `depositFromParent` on all child vaults in a loop (via `_distributeToChildren` and `_rebalanceToWeights`). `CarthaVault.depositFromParent` is protected by `whenNotPaused`.

If a single child vault is paused, `depositFromParent` reverts. Since `CarthaParentVault` does not check the paused state before calling, the revert propagates, causing the entire parent transaction to fail.

Impact

All `deposit`, `rebalance`, and `removePool` operations in `CarthaParentVault` are blocked indefinitely while any single child vault remains paused.

Recommendation

Modify the distribution loop in `CarthaParentVault` to check if a child is paused before attempting to deposit.

Status

Resolved

[M-02] CarthaParentVault#RemovePool - RemovePool Reverts When Miners Hold Shares in Child Vault

Description

The `CarthaParentVault` manages a list of child pools (`CarthaVault` instances) and allows admins to remove a pool via `removePool(...)`. As part of removal, the parent attempts to withdraw "all funds" from the child vault by using the child's `totalValueLocked()` as the withdrawal amount.

However, a `CarthaVault` is a share-based vault: both the parent vault and miners can hold shares. Therefore, `totalValueLocked()` represents the assets backing all share holders, not just the assets attributable to the parent vault's shares.Attempting to withdraw the full TVL from the parent will revert once any non-parent

Vulnerability Details

In `CarthaParentVault.removePool(...)`, the parent withdraws the child's full TVL:

```
uint256 vaultTVL = ICarthaVault(vault).totalValueLocked();

if (vaultTVL > 0) {

    ICarthaVault(vault).withdrawToParent(vaultTVL);

}
```

But in `CarthaVault.withdrawToParent(...)`, the requested amount is converted into shares and the caller must own at least those shares:

```
// Convert requested assets to shares (rounding up)

shares = (amount * totalShares + totalAssets_ - 1) / totalAssets_;

// @audit Reverts if parent does not own enough shares for `amount`

if (balanceOf(msg.sender) < shares) revert InsufficientShares();
```

If `amount == totalAssets_` (i.e., the parent requests the full TVL), then:

- `shares = ceil(totalAssets_ * totalShares / totalAssets_) = totalShares`

So the call requires the parent to own `totalShares` shares (100% ownership). If miners hold any shares, the parent's share balance is strictly less than `totalShares`, and `withdrawToParent(...)` reverts with `InsufficientShares()`. This revert happens before the function can "cap" the transfer by the vault's token balance, so it cannot be avoided by liquidity/rounding edge cases.

Impact

Once miners deposit into a child vault (or otherwise hold shares), `removePool(...)` becomes non-functional for that pool because the "withdraw all funds" step always reverts. This can block operational actions such as deprecating/removing a pool from the parent vault's management set and can hinder administrative response during incidents where removing a pool would be desirable.

Recommendation

Withdraw only the assets attributable to the parent's share ownership rather than withdrawing the entire child TVL. For example, derive the parent-owned assets as:

- `parentShares = IERC20(vault).balanceOf(address(this))`
- `totalShares = IERC20(vault).totalSupply()`
- `vaultTVL = ICarthaVault(vault).totalValueLocked()`
- `withdrawAmount = (parentShares * vaultTVL) / totalShares`

Then call withdrawToParent(withdrawAmount) (if non-zero) before removing the pool.

Status

Resolved

[M-03] CarthaVault#lockTopUp - Pending top-ups are lost on forced exits

Description

Pending top-ups are permanently lost when miners are force-exited before regularization.

Vulnerability Details

`lockTopUp(...)` tracks new funds in `position.pendingAmount`, but both `forceExit(...)` and `forceExitUnderperformer(...)` only return `position.amount` and delete the entire position. If a miner is evicted before the keeper calls `regularizeBalances(...)`, those pending funds remain in the vault and are never returned to the owner.

```
function lockTopUp(bytes32 poolId_, uint256 amount) external whenNotPaused {

    // ...

    IERC20($.asset).safeTransferFrom(msg.sender, address(this), amount);

    _mint(msg.sender, shares);



       // @audit Amount goes to pending, requires regularizeBalances() to move to
position.amount

    position.pendingAmount += amount;

    // ...

}

function forceExit(

    address owner,

    bytes32 poolId_,

    bytes32[] calldata merkleProof

) external onlyRole(Roles.EJECTOR) {

    // ...

    Position storage position = $.positions[lockId];
```

```
    // @audit Only position.amount is returned, pendingAmount is ignored
    uint256 amount = position.amount;
    uint256 shares = _convertToShares(amount);

    delete $.positions[lockId];
    $.totalPositions--;
    $.totalLockedAmount -= amount;

    _burn(owner, shares);
    IERC20($.asset).safeTransfer(owner, amount);
    // ...
}
```

The same issue exists in `forceExitUnderperformer(...)` which also ignores `pendingAmount`.

Impact

Direct loss of user funds. Miners lose their pending top-up amounts if evicted before regularization, and the vault retains those assets. Because shares were minted for the pending top-up but only shares for `position.amount` are burned during forced exits, the share supply becomes overstated relative to assets, which skews share pricing and reduces shares minted for future deposits.

Recommendation

Include `pendingAmount` in forced exit payouts and burn the correct total shares:

```
function forceExit(
    address owner,
    bytes32 poolId_,
    bytes32[] calldata merkleProof
```

```
) external onlyRole(Roles.EJECTOR) {

    // ...

    Position storage position = $.positions[lockId];



    // Include both locked and pending amounts

    uint256 totalAmount = position.amount + position.pendingAmount;

    uint256 shares = _convertToShares(totalAmount);



    delete $.positions[lockId];

    $.totalPositions--;

    $.totalLockedAmount -= position.amount;



    _burn(owner, shares);

    IERC20($.asset).safeTransfer(owner, totalAmount);

    // ...

}
```

Apply the same fix to `forceExitUnderperformer(...)`, ensuring the penalty calculation also includes `pendingAmount`.

Status

Resolved

[M-04] CarthaVault#lockTopUp - release(...) can delete pending top-ups

Description

The `CarthaVault` contract allows miners to top up their existing positions through `lockTopUp(...)`, which adds funds to `pendingAmount` and mints shares immediately. These pending amounts are meant to be regularized into the main `position.amount` by a keeper bot calling `regularizeBalances(...)` at epoch boundaries. However, the `release(...)` function has a critical flaw: when a user performs a full withdrawal by releasing exactly `position.amount`, the function deletes the entire position struct from storage, which also destroys any non-zero `pendingAmount` without returning those funds or burning the corresponding shares.

This creates a permanent loss scenario where users who legitimately withdraw their full "available" balance lose their pending top-ups that were already paid for and have minted shares backing them.

Vulnerability Details

The vulnerability exists in the interaction between three functions:

1. `lockTopUp(...)` - Accepts USDC, mints shares, stores amount in `pendingAmount`:

```
function lockTopUp(bytes32 poolId_, uint256 amount) external whenNotPaused {

    CarthaVaultStorage storage $ = _getCarthaVaultStorage();

    bytes32 lockId = lockIdOf(msg.sender, poolId_);

    Position storage position = $.positions[lockId];

    if (!position.exists) revert PositionNotFound();

    uint256 shares = _convertToShares(amount);

    IERC20($.asset).safeTransferFrom(msg.sender, address(this), amount);

    _mint(msg.sender, shares); // @audit Shares minted immediately

    // @audit Amount goes to pending, will be regularized at next epoch

    position.pendingAmount += amount;

    position.cooldownEnds = uint64(block.timestamp + $.cooldownDuration);
```

```
}
```

2. `release(...)` - Only allows withdrawing up to `position.amount`, deletes position on full release:

```
function release(bytes32 poolId_, uint256 amount) external whenNotPaused {
    CarthaVaultStorage storage $ = _getCarthaVaultStorage();
    bytes32 lockId = lockIdOf(msg.sender, poolId_);
    Position storage position = $.positions[lockId];
    if (!position.exists) revert PositionNotFound();
    // @audit Check only validates against position.amount, not pendingAmount
    if (amount == 0 || amount > position.amount) revert InvalidAmount();
    uint256 shares = _convertToShares(amount);
    bool isFullRelease = (amount == position.amount);
    if (isFullRelease) {
        // @audit This deletes the entire struct, including pendingAmount
        delete $.positions[lockId];
        $.totalPositions--;
    } else {
        position.amount -= amount;
    }
    $.totalLockedAmount -= amount;
    _burn(msg.sender, shares);
    IERC20($.asset).safeTransfer(msg.sender, amount);
}
```

3. `regularizeBalances(...)` - Moves `pendingAmount` to `amount`:

```
function regularizeBalances(address[] calldata owners) external
onlyRole(Roles.KEEPER_BOT) {
```

```
    CarthaVaultStorage storage $ = _getCarthaVaultStorage();

    for (uint256 i = 0; i < owners.length; i++) {

        bytes32 lockId = lockIdOf(owners[i], $.poolId);

        Position storage position = $.positions[lockId];

        if (position.exists && position.pendingAmount > 0) {

            uint256 pending = position.pendingAmount;

            position.amount += pending;

            position.pendingAmount = 0;

            $.totalLockedAmount += pending;

        }

    }

}
```

Attack scenario:

1. Miner locks 100,000 USDC via `lock(...)` → position.amount = 100,000, receives 100,000e18 shares
2. Miner tops up 50,000 USDC via `lockTopUp(...)` → `position.pendingAmount` = 50,000, receives additional 50,000e18 shares (total 150,000e18 shares)
3. Lock period and cooldown expire
4. Before keeper calls `regularizeBalances(...)`, miner calls `release(poolId, 100_000e6)` to withdraw everything they "see"
5. Contract deletes entire position including `pendingAmount = 50,000`
6. Miner receives only 100,000 USDC back but still holds 150,000e18 shares
7. The 50,000 USDC in `pendingAmount` is now permanently stuck in the contract with no position to claim it

Impact

Users lose the entire `pendingAmount` (could be substantial if multiple top-ups occurred)

Recommendation

Auto-regularize on release: Add a helper function that automatically moves pending to main amount before processing release, ensuring consistency.

Status

Resolved

[M-05] CarthaParentVault#_rebalanceToWeights - Rebalance compares parent targets to full child TVL

Description

The `CarthaParentVault` implements a rebalancing mechanism in `_rebalanceToWeights(...)` that shifts liquidity between child vaults to maintain target allocations. The function calculates `targetAllocation` using `totalAssets()`, which correctly represents the parent's proportional ownership. However, it compares this against `currentAllocation` which uses the full child TVL including miner deposits that the parent does not own.

This mismatch causes the rebalance logic to attempt withdrawals that exceed the parent's actual share, resulting in reverts whenever miners hold shares in child vaults.

Vulnerability Details

In `_rebalanceToWeights(...)` (lines 255-301):

```
uint256 vaultTVL = ICarthaVault(vault).totalValueLocked();

uint256 withdrawAmount = (amount * vaultTVL) / totalAssets_;
```

- `totalAssets_` correctly represents the parent's share (e.g., 10 USDC)
- `targetAllocation` is derived from parent's assets (e.g., 10 USDC with 100% weight)
- `currentAllocation` uses full child TVL (e.g., 100 USDC) including miner deposits

When `targetAllocation < currentAllocation` (`10 < 100`), the function attempts to withdraw the difference (90 USDC). The child vault's `withdrawToParent(...)` converts this to shares:

```
shares = (amount * totalShares + totalAssets_ - 1) / totalAssets_;

if (balanceOf(msg.sender) < shares) revert InsufficientShares();
```

The parent cannot provide 90 shares when it only owns 10, causing a revert.

Concrete Example:

- Child vault: 100 USDC TVL, 100 total shares
- Parent owns: 10 shares (10%)
- Miners own: 90 shares (90%)

Rebalance calculation:

1. `totalAssets()` = 10 USDC (parent's share)
2. `targetAllocation` = 10 USDC (100% weight)
3. `currentAllocation` = 100 USDC (full TVL)
4. `withdrawAmount` = 100 - 10 = 90 USDC
5. `withdrawToParent`(90) requires 90 shares
6. Parent has 10 shares → `REVERTS`

Impact

Rebalancing is completely non-functional when miners hold shares in any child vault. Since miners depositing is expected behavior, this effectively disables the rebalancing mechanism in production. The protocol cannot maintain target weights or respond to utilization changes.

Recommendation

Calculate `currentAllocation` using the parent's proportional share, matching the logic in `totalAssets()`:

```
function _rebalanceToWeights() internal returns (uint256 totalShifted) {

    ParentVaultStorage storage $ = _getParentVaultStorage();

    uint256 totalAssets_ = totalAssets();

    if (totalAssets_ == 0) return 0;

    // Calculate total weight

    uint256 totalWeight;
```

```
    for (uint256 i = 0; i < $.vaultList.length; i++) {
        address vault = $.vaultList[i];
        if ($.pools[vault].isActive && !ICarthaVault(vault).paused()) {
            totalWeight += $.pools[vault].weight;
        }
    }
    if (totalWeight == 0) return 0;
    // Rebalance each vault to its target allocation
    for (uint256 i = 0; i < $.vaultList.length; i++) {
        address vault = $.vaultList[i];
        if (!$.pools[vault].isActive || ICarthaVault(vault).paused()) continue;
        uint256 targetAllocation = (totalAssets_ * $.pools[vault].weight) / totalWeight;
        // Calculate parent's current allocation (not full TVL)
        uint256 parentShares = IERC20(vault).balanceOf(address(this));
        uint256 childTotalSupply = IERC20(vault).totalSupply();
        uint256 childTVL = ICarthaVault(vault).totalValueLocked();
        uint256 currentAllocation = (childTotalSupply > 0 && parentShares > 0)
            ? (parentShares * childTVL) / childTotalSupply
            : 0;
        if (targetAllocation > currentAllocation) {
            // Need to deposit more
            uint256 depositAmount = targetAllocation - currentAllocation;
            uint256 available = IERC20(asset()).balanceOf(address(this));
            if (depositAmount > available) depositAmount = available;
            if (depositAmount > 0) {
                IERC20(asset()).forceApprove(vault, depositAmount);
```

```
                ICarthaVault(vault).depositFromParent(depositAmount);
                IERC20(asset()).forceApprove(vault, 0);
                totalShifted += depositAmount;
            }
        } else if (targetAllocation < currentAllocation) {
            // Need to withdraw
            uint256 withdrawAmount = currentAllocation - targetAllocation;
            if (withdrawAmount > 0) {
                ICarthaVault(vault).withdrawToParent(withdrawAmount);
                totalShifted += withdrawAmount;
            }
        }
    }
}
```

Status

Resolved

[M-06] CarthaParentVault#_withdrawFromChildren - Parent withdrawals can revert when assets are spread across children

Description

The `CarthaParentVault` is an ERC4626 aggregator that manages liquidity across multiple child vaults. When users withdraw, the parent calls `_withdrawFromChildren(...)` to pull assets proportionally from each child. However, there is a mismatch between how totalAssets() and `_withdrawFromChildren(...)` calculate values.

`totalAssets()` correctly computes the parent's proportional share of each child vault using `(parentShares * childTVL) / childTotalSupply`. In contrast, `_withdrawFromChildren(...)` uses the full child TVL in its withdrawal calculation formula, which inflates the per-vault withdrawal request relative to the parent's actual ownership.

Vulnerability Details

In `_withdrawFromChildren(...)` (lines 479-498), the withdrawal amount per vault is calculated as:

```
uint256 vaultTVL = ICarthaVault(vault).totalValueLocked();

uint256 withdrawAmount = (amount * vaultTVL) / totalAssets_;
```

This formula uses vaultTVL (the full child TVL including miner deposits) against `totalAssets_` (only the parent's share). When the parent owns a minority of a child vault's shares, the calculated `withdrawAmount` can exceed what the parent is entitled to withdraw.

Although line 492 caps `withdrawAmount` to remaining, the capped value can still exceed the parent's share in that specific vault. When `withdrawToParent(...)` is called on the child vault (line 495), it converts the requested assets to shares:

```
shares = (amount * totalShares + totalAssets_ - 1) / totalAssets_;

if (balanceOf(msg.sender) < shares) revert InsufficientShares();
```

If the parent lacks sufficient shares, the transaction reverts.

Concrete Example:

- Child Vault 1: 100 USDC TVL, parent owns 10 shares out of 100 (10%)
- Child Vault 2: 100 USDC TVL, parent owns 10 shares out of 100 (10%)
- totalAssets() = 20 USDC (parent's share)

User withdraws 15 USDC:

1. withdrawAmount = (15 * 100) / 20 = 75 USDC
2. Capped to remaining = 15 USDC
3. withdrawToParent(15) requires 15 shares
4. Parent only has 10 shares → REVERTS

Impact

Users cannot withdraw funds when the requested amount exceeds the parent's share in the first child vault. This blocks legitimate withdrawals even though the parent owns sufficient total assets across all children. The issue affects any multi-vault configuration where assets are distributed and miners hold shares.

Recommendation

Calculate withdrawals based on the parent's share of each child vault rather than the full child TVL:

```
function _withdrawFromChildren(uint256 amount) internal {

    ParentVaultStorage storage $ = _getParentVaultStorage();

    uint256 remaining = amount;

    uint256 totalAssets_ = totalAssets();

    for (uint256 i = 0; i < $.vaultList.length && remaining > 0; i++) {

        address vault = $.vaultList[i];

        if (!$.pools[vault].isActive) continue;
```

```
        // Calculate parent's share of this vault
        uint256 parentShares = IERC20(vault).balanceOf(address(this));
        uint256 childTotalSupply = IERC20(vault).totalSupply();
        uint256 childTVL = ICarthaVault(vault).totalValueLocked();
        if (childTotalSupply == 0 || parentShares == 0) continue;
        uint256 parentShareValue = (parentShares * childTVL) / childTotalSupply;
        uint256 withdrawAmount = (amount * parentShareValue) / totalAssets_;
        if (withdrawAmount > remaining) withdrawAmount = remaining;
        if (withdrawAmount > parentShareValue) withdrawAmount = parentShareValue;
        if (withdrawAmount == 0) continue;
        uint256 withdrawn = ICarthaVault(vault).withdrawToParent(withdrawAmount);
        remaining -= withdrawn;
    }
}
```

Status

Resolved

Low

[L-01] LibDiamond.sol - Incorrect Storage Slot Calculation

Description

Storage slot constants in multiple contracts do not match the ERC-7201 formula, and `LibDiamond.sol` uses an inconsistent storage pattern.

Vulnerability Details

Multiple contracts contain hardcoded storage slot constants that do not match the values derived from the ERC-7201 formula `keccak256(uint256(keccak256(id)) - 1) & ~bytes32(uint256(0xff))`, despite annotations or code comments indicating adherence to this standard. Additionally, `LibDiamond.sol` deviates from the project's standard storage layout pattern.

1. CarthaParentVault.sol:
    - ID: `cartha.storage.ParentVault`
    - Hardcoded: `0x8c65c6f8c20b3cb06c1a6b8e6b9e5c5e1f4d3c2b1a0908070605040302010000`
    - Calculated: `0x132a1344191806d6acd57512ba0c4193d7f59e510e17f64a6c527c52607be700`
2. CarthaVault.sol:
    - ID: `cartha.storage.CarthaVault`
    - Hardcoded: `0x742e9e6b4d8c6e4d8e6b4d8c6e4d8e6b4d8c6e4d8e6b4d8c6e4d8e6b4d8c6e00`
    - Calculated: `0xe912e8bf36bf83660d227f6af14ede2c91a4dbb2e01a1a24b1d461b36bc30a00`
3. CarthaVaultFactory.sol:
    - ID: `cartha.storage.CarthaVaultFactory`
    - Hardcoded: `0x8f8c8e7e0c5c4d4e3f3a2b1c0d9e8f7a6b5c4d3e2f1a0b9c8d7e6f5a4b3c2d00`
    - Calculated: `0xdb1962c4a5690592ca7465b77ca9c0256f84cb63342b7979f63bc5799a61d100`
4. LibDiamond.sol:

- Used: `keccak256("cartha.diamond.storage")`
- Issue: Inconsistent with the ERC-7201 namespaced storage pattern used in other system components, potentially complicating layout management.

Proof of Concept

The following `chisel` session calculates the correct values using the standard formula:

```
> chisel

➜ keccak256(abi.encode(uint256(keccak256("cartha.storage.ParentVault")) - 1)) & ~bytes32(uint256(0xff))

Hex: 0x132a1344191806d6acd57512ba0c4193d7f59e510e17f64a6c527c52607be700

➜ keccak256(abi.encode(uint256(keccak256("cartha.storage.CarthaVault")) - 1)) & ~bytes32(uint256(0xff))

Hex: 0xe912e8bf36bf83660d227f6af14ede2c91a4dbb2e01a1a24b1d461b36bc30a00

➜ keccak256(abi.encode(uint256(keccak256("cartha.storage.CarthaVaultFactory")) - 1)) & ~bytes32(uint256(0xff))

Hex: 0xdb1962c4a5690592ca7465b77ca9c0256f84cb63342b7979f63bc5799a61d100
```

Impact

Storage collisions might occur if the hardcoded slots overlap with other variables. Mismatched storage slots violate the intended ERC-7201 standard, which can cause issues with tooling, upgrades, and state consistency.

Recommendation

Update the constants in each file to their correctly calculated ERC-7201 values:

- CarthaParentVault.sol: `0x132a1344191806d6acd57512ba0c4193d7f59e510e17f64a6c527c52607be700`
- CarthaVault.sol: `0xe912e8bf36bf83660d227f6af14ede2c91a4dbb2e01a1a24b1d461b36bc30a00`
- CarthaVaultFactory.sol: `0xdb1962c4a5690592ca7465b77ca9c0256f84cb63342b7979f63bc5799a61d100`

For LibDiamond.sol, adopt the ERC-7201 standard for consistency (e.g., id `cartha.storage.Diamond`) unless the current pattern is intentional for specific compatibility reasons.

Status

Resolved

[L-02] CarthaVault#rescueFunds - Rescue Funds Rug Pull

Description

The `rescueFunds` function in `CarthaVault` allows the `RESCUER` role to withdraw any token, including the underlying asset, bypassing all protocol constraints.

Vulnerability Details

The function lacks a check `if (token == asset()) revert;`, enabling the transfer of user deposits via `safeTransfer` to any recipient.

Impact

A `RESCUER` can instantly drain the entire vault TVL creating a rug pull mechanism.

Recommendation

Add `if (token == asset()) revert CannotRescueAsset();` to rescueFunds.

Status

Resolved

[L-03] CarthaParentVault - Missing Decimals Offset Configuration

Description

The `CarthaParentVault` relies on the default OpenZeppelin v5 implementation of `_decimalsOffset()`, which returns `0`. While the underlying `ERC4626` implementation contains basic mitigations (adding +1 to the denominator) to prevent profitable inflation attacks, the lack of a sufficient decimals offset leaves the vault susceptible to "Zero Share" Denial of Service (DoS) attacks.

Vulnerability Details

When `_decimalsOffset()` is 0, the vault does not use "virtual shares" to buffer against exchange rate manipulation.

Formula: shares = `assets * (totalSupply + 1) / (totalAssets + 1)`.

If an attacker can manipulate `totalAssets` to be significantly larger than `totalSupply` (via donation), lawful deposits can round down to 0 shares. Although this is generally unprofitable for the attacker in v5, it allows for a griefing attack where an attacker burns funds to ensure subsequent users lose their deposits.

Proof of Concept

```
function testInflationAttack() public {

        vm.startPrank(attacker);

        token.mint(attacker, 1_000_000e18);

        token.approve(address(parentVault), 1);

        parentVault.deposit(1, attacker);

        token.transfer(address(parentVault), 1_000_000e18);

        vm.stopPrank();

        vm.startPrank(user1);

        token.approve(address(parentVault), 100e18);

        uint256 shares = parentVault.deposit(100e18, user1);

        console.log("H-01 Victim Shares:", shares);
```

```
        assertEq(shares, 0, "Victim should have 0 shares");

        vm.stopPrank();

    }
```

Impact

Low. The attack is economically irrational for the attacker (they lose funds) but poses a griefing risk. Implementing a proper offset is a standard hardening practice for ERC4626 vaults.

Recommendation

Override `_decimalsOffset()` in `CarthaParentVault.sol` to return a value of `9` (assuming 18 decimal shares and 6 decimal underlying). This introduces a virtual supply of `10^9`, effectively neutralizing the rounding-to-zero risk for any realistic donation amount.

Status

Resolved

[L-04] Diamond - NO eth recovery path

Description

The `Diamond` proxy has a `receive()` function and a payable `fallback()`, which means the Diamond address can

Vulnerability Details

The Diamond itself accepts ETH via `receive()`:

- A plain ETH transfer (empty calldata) succeeds and increases `address(diamond).balance`.
- The currently deployed facets do not have any `payable` functions, so sending `msg.value > 0` to a routed function selector will revert at the facet-level (non-payable function call).

Impact

Accidentally sent ETH may sit idle on the Diamond until the owner adds a recovery facet (or otherwise upgrades).

Recommendation

If ETH is not intended to ever be held by the Diamond:

- Make `receive()` revert (or remove it), and consider making `fallback()` non-payable to reject `msg.value`.

If ETH recovery is desired:

- Add an explicit owner-only (or admin-only) sweep function in a facet, so any ETH held by the Diamond can be recovered without relying on ad-hoc upgrades.

Status

Resolved

[L-05] CarthaVault#_update - Users can avoid forced exit penalties by transferring shares after lock expiry

Vulnerability Details

The `CarthaVault` allows miners to be forcefully exited with penalties through the `forceExitUnderperformer(...)` function when they are marked as underperformers. The transfer hook only prevents share transfers while the position is locked. Once the lock period expires, miners can freely transfer their shares to another address they control. The position data remains tied to the original owner address, and when the protocol attempts to enforce penalties via `forceExitUnderperformer(...)`, the burn operation fails because the original owner no longer holds sufficient shares.

This allows underperforming miners to avoid performance penalties while retaining the full economic value of their position through a secondary address.

```
function _update(address from, address to, uint256 value) internal override {

    if (from != address(0)) {

        CarthaVaultStorage storage $ = _getCarthaVaultStorage();

        bytes32 lockId = lockIdOf(from, $.poolId);

        Position storage position = $.positions[lockId];

        if (position.exists) {

            bool locked =

                block.timestamp < position.start + uint256(position.lockDays) * 1 days;

            // @audit Only blocks transfers during lock period

            if (locked) {

                revert PositionLocked();

            }

        }

    }
```

```
    super._update(from, to, value);
}
function forceExitUnderperformer(
    address owner,
    bytes32 poolId_,
    bytes32[] calldata merkleProof
) external onlyRole(Roles.EJECTOR) {
    // ...
    uint256 penalty = (amount * $.performancePenalty) / 10000;
    uint256 amountAfterPenalty = amount - penalty;
    delete $.positions[lockId];
    $.totalPositions--;
    $.totalLockedAmount -= amount;
    // @audit Burns from original owner who may have transferred shares away
    _burn(owner, shares);
    IERC20($.asset).safeTransfer(owner, amountAfterPenalty);
    // ...
}
```

Impact

Underperforming miners can avoid performance penalties by transferring their shares to a secondary address after the lock expires. The protocol cannot enforce penalties on these users, and the position remains in storage indefinitely, causing storage bloat and accounting inconsistencies in `totalPositions` and `totalLockedAmount`. The penalty amount that should be retained by the protocol is lost.

Recommendation

Consider the following approaches,

1. Prevent transfers while position exists: Modify the transfer hook to check `position.exists` rather than just the lock status

Status

Acknowledged

QA

[Q-01] CarthaParentVault - Unused Pause Functionality

Description

The `CarthaParentVault` contract inherits and initializes `PausableUpgradeable` but fails to expose pause logic or apply it to important functions.

Vulnerability Details

No external functions exist to trigger `_pause()` or `_unpause()`, and the `whenNotPaused` modifier is completely absent from all state-changing functions.

Impact

Admins cannot halt deposits or withdrawals during emergencies, preventing a "circuit breaker" response to exploits.

Recommendation

Implement admin-only `pause`/`unpause` functions and add the `whenNotPaused` modifier to `_deposit`, `_withdraw`, and `shiftLiquidity`.

Status

Resolved

[Q-02] CarthaParentVault#rebalance - Function Has Empty Implementation

Description

The `rebalance()` function in `CarthaParentVault.sol` contains a `TODO` and lacks core logic. It updates `lastRebalanceTime` and emits Rebalanced but performs no actual fund redistribution.

Vulnerability Details

The function body is incomplete:

```
// TODO: Implement fund shifting based on target weights
```

This is a no-op that misleads keepers into believing rebalancing occurred.

Impact

Protocol cannot auto-maintain its target allocation strategy, causing strategy drift over time.

Recommendation

Call the existing `_rebalanceToWeights()` internal function which already implements the correct logic.

Status

Resolved

[Q-03] CarthaParentVault - Approval Not Cleared After Child Vault Deposits

Description

`CarthaParentVault` grants approvals to child vaults for `deposits` but fails to reset the allowance to zero immediately after usage.

Vulnerability Details

In `shiftLiquidity`, `_rebalanceToWeights`, and `_distributeToChildren`, approvals are granted to child vaults. If `depositFromParent` consumes less than the approved amount (e.g., due to rounding), residual allowances remain active.

Impact

This is a best practice issue. Ensuring zero allowance after transfers prevents residual approval accumulation and enforces strict least privilege.

Recommendation

Explicitly reset approval to zero after the transfer to confirm no residual allowance remains.

Status

Resolved

[Q-04] CarthaVault#setEvictionRoot - Missing zero merkle root validation

Description

The `setEvictionRoot` and `setActiveMinersRoot` functions lack validation for zero values (`bytes32(0)`).

Vulnerability Details

The admin functions allow setting critical `Merkle` roots to zero without checks.

```
function setEvictionRoot(bytes32 newRoot) external onlyRole(Roles.ADMIN) {

    CarthaVaultStorage storage $ = _getCarthaVaultStorage();

    $.evictionRoot = newRoot;

}
```

Impact

Setting roots to zero bricks `forceExit` and miner validation mechanisms, causing Denial of Service until corrected.

Recommendation

Add a check: `if (newRoot == bytes32(0)) revert InvalidRoot();`.

Status

Resolved

[Q-05] CarthaVaultFactory#addChildToParent - Missing validation for same asset between parent and child

Description

Factory does not validate parent/child asset match.

Vulnerability Details

`addChildToParent(...)` links child vaults to parents without verifying asset equality.

```
function addChildToParent(address parent, address child, uint256 weight, uint256 maxAllocation) external onlyRole(Roles.ADMIN) {

    // ... checks ...

    // @audit Missing asset equality check

    ICarthaParentVault(parent).addPool(child, weight, maxAllocation);

}
```

Impact

Linking incompatible assets causes parent vault operations to revert (DoS) or leads to accounting errors.

Recommendation

Require `ICarthaParentVault(parent).asset() == ICarthaVault(child).asset()`.

Status

Resolved

Centralisation

The Cartha project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Cartha project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au